

Vision-Scan: Neural Vision Pipeline for Real-Time PCB Anomaly Detection

Kokou Adje

*College of Computing and Software Engineering,
Kennesaw State University*

`kadje@students.kennesaw.edu`

Abstract—Printed Circuit Boards (PCBs) anchor modern electronics, yet traditional inspection remains slow, subjective, and misses over twenty percent of defects on high-volume lines. This work presents Vision-Scan, a deep learning pipeline for automated PCB defect detection across six classes: open, short, mousebite, spur, copper, and pin-hole. The system applies synthetic augmentation — rotation, flipping, brightness shifts, noise injection, and Gaussian blur — to expand a 1,050-image training set four-fold while preserving defect geometry, fine-tunes a YOLOv8s detector using transfer learning from COCO weights, and exports the trained model to ONNX for portable deployment. Trained for 50 epochs with early stopping on Kaggle's NVIDIA T4 GPU, Vision-Scan reaches 0.983 mAP@0.5 on the held-out DeepPCB test split, with per-class mAP@0.5 ranging from 0.965 (short) to 0.993 (pin-hole). Mean precision is 0.979 and mean recall is 0.948. PyTorch inference runs at 99 frames per second on T4, comfortably exceeding the conveyor-speed threshold for production inspection. The confusion matrix shows minimal class-to-class confusion; the dominant residual error is missed detection rather than misclassification, suggesting that further gains would come from improved recall on small defects rather than from sharper class boundaries.

Index Terms—PCB inspection, defect detection, YOLO, deep learning, transfer learning, synthetic augmentation, ONNX, edge deployment.

I. INTRODUCTION

Printed Circuit Boards (PCBs) form the nervous system of nearly every modern electronic device, spanning consumer electronics, medical instrumentation, aerospace avionics, telecommunications infrastructure, and automotive control units [1]. As manufacturers push toward thinner substrates and higher component densities, the physical features that must be inspected have shrunk well below what unaided human vision can reliably resolve. Defects such as micro-fractures, missing components, solder bridges, open circuits, mousebites, and spurs propagate into functional failures that range from warranty returns to safety-critical incidents when shipped boards enter the field.

Two inspection strategies dominate current practice. Manual microscope inspection depends entirely on operator skill and concentration; published estimates place the miss

rate above twenty percent on high-throughput lines, with throughput itself capped by human fatigue [1]. Automated Optical Inspection (AOI) systems replace the human eye with structured lighting and rule-based image comparison, but they remain brittle under illumination variation, require painstaking golden-template calibration for each board revision, and generate high false-positive rates that force operators back into a secondary manual review loop [2].

Deep learning offers a path past both limitations. Convolutional neural networks trained end-to-end learn feature extractors tailored to the defect distribution rather than relying on hand-designed filters. Among detection architectures, the YOLO family has emerged as a particularly strong fit for PCB inspection because it formulates detection as a single regression pass over a grid of anchors, delivering competitive accuracy at frame rates compatible with production conveyor speeds [3], [4]. Faster R-CNN and DETR variants achieve slightly higher precision on some benchmarks but pay for it in latency that production lines cannot absorb.

Two practical obstacles still block deployment. First, labeled PCB defect datasets are small relative to the hundreds of thousands of images typical in general-purpose detection benchmarks; manufacturers treat defective boards as intellectual property and rarely release them. Second, factory edge hardware imposes tight memory and compute budgets that exclude larger backbones unless models are quantized and runtime-optimized for the target accelerator [5], [6].

The economic stakes reinforce the technical ones. PCB defects discovered after final assembly cost roughly ten times more to repair than defects caught at inspection, and defects that reach field deployment cost a hundred times more when warranty, recall, and reputational damage are included. A detection system that reduces miss rate even by a few percentage points translates directly into substantial savings for a contract manufacturer.

This paper presents Vision-Scan, a pipeline designed to address both constraints directly. The contributions are: (1) a synthetic augmentation stage grounded in classical image processing that expands the effective training set four-fold without manual re-labeling; (2) a transfer-learning protocol for fine-tuning YOLOv8 on the DeepPCB dataset that reaches 0.983 mAP@0.5 — comparable to published baselines on this benchmark; and (3) an ONNX export path that produces a portable deployed model. The work is fully reproducible from public dependencies on a public dataset, and runs end-to-end on a single Kaggle GPU notebook in under three hours.

The remainder of the paper is organized as follows. Section II reviews related work. Section III details the pipeline design. Section IV describes implementation. Section V reports quantitative results. Section VI concludes and outlines future work.

II. RELATED WORK

A. Classical Inspection and Traditional Computer Vision

Early machine vision for PCB inspection relied on template matching, edge-based comparison, and connected-component analysis applied to the difference between a test image and a known-good reference [1]. Czimmermann et al. survey this generation of methods and document their central weakness: rigid sensitivity to lighting geometry, component placement tolerance, and reference image quality. Texton histograms and Bag-of-Features representations extended the expressive power of traditional pipelines by summarizing local patch statistics into global descriptors, but they remained handcrafted and brittle when deployed on board revisions not seen during calibration [8]. Part-based models attempted to close this gap by modeling spatial relationships between learned part filters, yet they required careful feature engineering per defect category.

Commercial AOI systems represent the production-deployed evolution of these classical methods. They typically combine multi-angle structured lighting, registered template comparison, and rule-based defect classifiers tuned per board layout. The performance of an AOI system depends heavily on the quality of the golden template and the stability of the imaging conditions; both are difficult to maintain across shift changes, lamp aging, and seasonal temperature variations on the factory floor. The high false-positive rates documented in the literature are a direct consequence of these sensitivities, and the secondary manual review loop they trigger erodes much of the throughput advantage AOI was supposed to provide.

B. Deep Learning for Surface Defect Detection

The broader surface-defect literature has converged on two architectural families. Segmentation-based approaches, exemplified by Tabernik et al. [5], treat defect detection as dense pixel classification and excel on datasets with large, visually distinctive anomalies. Detection-based approaches frame the problem as bounding-box regression and dominate benchmarks where defects are small, numerous, and spatially sparse. Liu et al. [6] review convolutional methods applied specifically to metal surface inspection and report that detection pipelines generally outperform segmentation when the target classes are well-defined and the dataset includes reliable box annotations.

Two-stage detectors such as Faster R-CNN achieve strong precision by decoupling region proposal from classification, but their computational profile is poorly suited to edge deployment. Single-stage detectors — SSD, RetinaNet, and the YOLO family — sacrifice a small accuracy margin for substantially reduced latency and predictable inference time that matches the constant-throughput requirements of conveyor-speed inspection.

C. YOLO for PCB Inspection

Adibhatla et al. [2] demonstrated that the YOLO architecture, originally designed for natural-scene detection, transfers effectively to PCB defect identification when fine-

tuned on domain-specific data. Their YOLOv2-based system reached mean average precision above 98 percent on a proprietary dataset. The Ultralytics YOLOv5 release [4] standardized PyTorch-based training and export to ONNX, and the YOLOv6 family [10] introduced industrial-grade latency optimizations. Redmon and Farhadi's YOLOv3 paper [3] remains the canonical reference for the multi-scale prediction heads that make the architecture effective at detecting defects of widely varying size, which matters directly for PCB inspection since pin-hole defects occupy a handful of pixels while copper excess can span hundreds.

YOLOv8 [13] refines the anchor-free detection head introduced in YOLOX, adopts a CSPDarknet backbone with path aggregation, and integrates distribution focal loss for box regression. The architecture emits predictions at three feature scales — typically corresponding to strides of 8, 16, and 32 pixels — which gives the network natural sensitivity to the wide range of defect sizes present in PCB imagery. The smaller YOLOv8s variant used in this work has 11.1 million parameters, fits comfortably in T4 memory, and exports cleanly to ONNX without requiring custom operators.

D. Synthetic Data Augmentation

The data-scarcity problem in industrial defect detection has motivated a range of synthetic augmentation approaches. Classical geometric and photometric augmentations — rotation, flipping, brightness jitter, noise injection, and blur — provide the baseline and are directly traceable to the image processing techniques catalogued by Szeliski [8]. The Albumentations library [11] systematizes these operators and guarantees bounding-box consistency under geometric transforms, which is essential for detection tasks where mislabeled augmented boxes silently harm training. This work restricts itself to classical augmentation because the method is well-understood, computationally cheap, and its effect on bounding-box geometry is verifiable. More recent generative approaches — copy-paste synthesis and diffusion-based defect generation — offer higher diversity at the cost of additional training infrastructure and validation overhead, and are noted as future work.

E. Anomaly Detection and Benchmark Datasets

Chandola et al. [7] provide the foundational taxonomy of anomaly detection approaches across supervised, semi-supervised, and unsupervised regimes, a framing that directly informs how defect detection is cast in this work. Vision-Scan operates in the supervised regime because labeled bounding boxes are available through DeepPCB. On the dataset side, DeepPCB, introduced by Tang et al. [9], provides 1,500 paired template / test images with pixel-accurate defect annotations across six classes — open, short, mousebite, spur, copper, and pin-hole — and has become the standard public benchmark for PCB detection research. The dataset's size is small by the standards of general object detection but representative of the labeled data available to most contract manufacturers, which makes results obtained

on DeepPCB more transferable to real production settings than results obtained on hundred-thousand-image synthetic benchmarks.

F. Edge Deployment

Factory-floor deployment requires inference latency compatible with conveyor throughput, typically ten to thirty frames per second. ONNX Runtime [12] dominates portable deployment and supports CPU, CUDA, and OpenVINO targets through a single model format. Post-training FP16 quantization typically reduces latency by a factor of two with negligible accuracy loss, and INT8 quantization can deliver another two-fold speedup at the cost of a small calibration step. The Ultralytics export pipeline automates the conversion path used in this work. NVIDIA TensorRT offers a CUDA-specific alternative with higher peak throughput but tighter coupling to the deployment hardware; this work uses ONNX Runtime in the interest of portability.

G. Small Object Detection

PCB defects often occupy a small fraction of the image area, which places them in the small-object detection regime where general detection benchmarks tend to underperform. Lin et al. [17] introduced focal loss specifically to address the foreground-background class imbalance that makes dense detectors struggle on small objects, and the loss formulation has since been incorporated into most modern detector architectures. Feature Pyramid Networks combine high-resolution low-level features with semantically rich high-level features, giving small-object predictions access to both spatial detail and abstract context; this is the architectural principle behind the multi-scale prediction heads in the YOLO family. He et al. [18] showed that residual connections allow much deeper backbones to train stably, which indirectly benefits small-object detection because the deeper feature hierarchy provides finer spatial resolution at the relevant prediction scales.

H. Two-Stage and Transformer-Based Detectors

Two-stage detectors and transformer-based detectors represent the two main alternatives to single-stage YOLO-style architectures. Faster R-CNN by Ren et al. [19] decouples detection into a region proposal network followed by a classification and regression head, achieving high accuracy at the cost of latency that often exceeds the budget available for production inspection. The Detection Transformer (DETR) by Carion et al. [20] eliminates anchor design and non-maximum suppression entirely by formulating detection as a set prediction problem solved with bipartite matching, but DETR variants typically require longer training schedules and larger datasets than the defect-detection domain offers. For PCB inspection specifically, the literature consistently favors single-stage convolutional detectors because their predictable per-frame latency matches the constant-rate requirements of conveyor-speed imaging systems.

I. Optimization and Training Stability

The optimizer used in this work, AdamW, was introduced by Loshchilov and Hutter [16] as a fix to the weight decay coupling problem in the original Adam optimizer; the decoupled formulation produces more reliable convergence on small datasets and has become the default choice for fine-tuning detector models. The cosine annealing learning rate schedule complements AdamW by providing smooth decay to a small final learning rate, which helps the model converge to a stable minimum during the late training epochs when mosaic augmentation is disabled. Together these choices reduce the hyperparameter search space substantially: the practitioner only needs to select an initial learning rate and a total number of epochs, with the schedule and optimizer handling the rest.

J. Cross-Dataset Generalization

MVTec AD by Bergmann et al. [14] is the most widely used public benchmark for industrial anomaly detection and provides pixel-level annotations across fifteen object and texture categories. Although this work evaluates only on DeepPCB to constrain scope, MVTec AD would be the natural target for cross-dataset evaluation because its imaging conditions and defect distribution differ from DeepPCB enough to expose generalization weaknesses. Microsoft COCO [15], used here only as the source of pretrained weights, also illustrates how scale of pretraining data drives transfer-learning effectiveness — the COCO-pretrained backbone transferred to PCB imagery without requiring tens of thousands of in-domain training images, which matches the data-efficient transfer behavior reported across the detection literature.

K. Gap Addressed by This Work

Prior PCB-specific studies tend to focus on a single benchmark and report only aggregate accuracy [2]. This work combines the three components needed for an end-to-end deployable system — synthetic augmentation, transfer-learned YOLOv8 fine-tuning, and ONNX export — and reports both detection accuracy and inference throughput on a public benchmark for reproducibility. The diagnostic depth provided by the confusion matrix decomposition (Section V-B) also goes beyond aggregate-only reporting and surfaces specific failure modes that aggregate mAP cannot reveal.

III. PROPOSED METHOD

A. Pipeline Overview

The Vision-Scan pipeline executes three sequential stages. The augmentation stage takes the original DeepPCB training split and emits an expanded training set with consistently transformed images and YOLO-format labels. The training stage consumes the augmented set, fine-tunes a COCO-pretrained YOLOv8s backbone, and writes out PyTorch weights plus a full set of evaluation metrics. The export stage converts the trained weights to ONNX format. Each stage produces artifacts consumed by the next, and each can be rerun independently.

The data flow from raw DeepPCB images to deployed model proceeds in five concrete steps: (1) the preparation script ingests DeepPCB and emits YOLO-format annotations split into train, validation, and test directories; (2) the augmentation script reads the train directory and writes an expanded train set; (3) the training script reads the expanded train set, validates against the original validation set every epoch, and writes weights and metrics to a run directory; (4) the evaluation script reads the trained weights and the test set and emits a JSON file with aggregate and per-class metrics; (5) the export script reads the trained weights and emits an ONNX file usable by any ONNX Runtime target. This decomposition mirrors the standard machine learning operations pattern of separated, versionable stages, which makes the pipeline straightforward to integrate into a continuous-integration system if it were taken into production.

The repository structure follows the same decomposition. A scripts directory holds the dataset preparation script, a src directory holds the five pipeline modules, and a configs directory holds the YOLO dataset YAML. This separation makes it possible to add support for a new dataset by writing only a new preparation script and a new YAML config, without touching the augmentation, training, evaluation, or export code. It also makes it possible to swap the detector — for example, replacing YOLOv8s with YOLOv8m or with a different architecture entirely — by changing only the model argument passed to the training script.

B. Stage 1 — Synthetic Augmentation

Six augmentation operators are applied, each corresponding to a transformation class grounded in classical image processing as catalogued by Szeliski [8]. Geometric operators include small-angle rotation in the range of plus or minus fifteen degrees, horizontal flip, and vertical flip; these sample the symmetries a board can present to a mounted inspection camera. Photometric operators include brightness and contrast jitter up to thirty percent, Gaussian noise with variance sampled between ten and fifty, and Gaussian blur with kernel size three to seven; these simulate the illumination drift and sensor noise typical of production environments.

Each operator is grounded in a specific failure mode of factory imaging. The rotation operator addresses small angular misalignments between the board and the camera mount that occur when boards seat imperfectly in the inspection fixture. The horizontal and vertical flip operators address the symmetry of board layouts; many PCBs have layouts that are approximately invariant under reflection, and exposing the model to flipped versions during training prevents it from learning a spurious orientation prior. The brightness and contrast operator addresses lamp aging and ambient lighting drift across shifts. The Gaussian noise operator addresses sensor read noise. The Gaussian blur operator addresses small focus errors that occur when board thickness varies across the inspection area.

All operators transform images and their bounding-box labels simultaneously through the Albumentations library [11], which guarantees geometric consistency between pixels and annotations. Each original image generates three augmented copies, each produced by one randomly selected operator. The training set grew from 1,050 originals to 4,194 total images after augmentation, with 56 augmentations skipped due to bounding-box validity constraints after geometric transformation.

C. Stage 2 — YOLOv8 Fine-Tuning

The detector is initialized from YOLOv8s weights pretrained on COCO. Transfer learning is preferred over training from scratch because the low-level features learned on COCO — edges, corners, oriented gradients — transfer cleanly to PCB imagery while the upper layers adapt to the defect distribution. The full network is trained at 640×640 input resolution with batch size 16, using AdamW optimization, an initial learning rate of 10^{-3} , and a cosine decay schedule. Early stopping is triggered if validation mAP fails to improve for thirty consecutive epochs, with a maximum of 100 epochs.

The detection loss is the standard YOLOv8 combination of distribution focal loss for box regression, binary cross-entropy for objectness, and cross-entropy for classification across the six DeepPCB classes. Each prediction emits a bounding box in pixel coordinates, a class label from the six-element taxonomy, and a confidence score.

D. Stage 3 — ONNX Export

The trained PyTorch weights are converted to the ONNX format with opset 12 and graph simplification enabled. ONNX runs on any platform with ONNX Runtime support, including CPU, CUDA, and OpenVINO targets, making the deployed model portable across the heterogeneous hardware typical of factory inspection systems. The exported model preserves the input resolution and output decoding used during training so downstream inference is drop-in compatible with the PyTorch reference.

IV. IMPLEMENTATION

A. Environment and Tooling

All experiments run on a Kaggle GPU notebook with NVIDIA T4 hardware (15 GB VRAM). The software stack includes PyTorch 2.10 with CUDA 12.8, Ultralytics 8.4.46 for YOLO training and export, Albumentations 1.3 for augmentation, ONNX 1.20 with ONNX Runtime 1.25 for portable inference, and OpenCV 4.8 for image I/O. Code is organized into a single repository with separate modules per pipeline stage, enabling independent reruns and clean unit-testing debugging.

B. Dataset Preparation

DeepPCB provides 1,500 paired template and test images at roughly 640×640 resolution. A preparation script walks the source directory, reads each image to determine pixel

dimensions, converts the native pixel-coordinate annotations into YOLO's normalized center-x, center-y, width, height format, and re-indexes the six-class labels to the zero-based numbering required by YOLO. Degenerate boxes with zero width or height are discarded. The resulting dataset is split 70-15-15 across train, validation, and test with a fixed random seed for reproducibility.

DeepPCB stores defective images with the suffix `_test.jpg` and matched defect-free templates with `_temp.jpg`. Annotations live in sibling folders named with the suffix `_not` at the same hierarchy level as the image folders, with filenames stripped of the `_test` suffix; the preparation script handles this layout explicitly. Annotations in the source dataset use one-indexed class IDs from one to six, which the preparation script remaps to zero-indexed labels matching the YOLO convention. After preparation, the splits contain 1,050 training images with 7,037 boxes, 225 validation images with 1,478 boxes, and 225 test images with 1,498 boxes.

C. Training Protocol

Training runs with the hyperparameters listed in Section III-C. Mixed-precision (FP16) training is enabled to fit the batch size into T4 memory. The Ultralytics trainer writes per-epoch metrics to `results.csv`, full plots including confusion matrix and precision-recall curve to the run directory, and both last and best weight checkpoints based on validation $\text{mAP}@0.5:0.95$. A cosine learning rate schedule decays from 10^{-3} to 10^{-5} across the planned 100 epochs.

Mosaic augmentation, which composes four training images into a single input, is active for most of training and disabled for the final 10 epochs to let the model converge on clean samples matching the evaluation distribution. Validation runs after every epoch on the 15 percent validation split; the best-mAP checkpoint is retained as the final model. The patience parameter is set to 30 epochs because pilot runs showed that mAP plateaus longer than this rarely recover on the small DeepPCB dataset.

D. Evaluation Protocol

Evaluation is reported on the held-out DeepPCB test split (15 percent of the original data, unseen during training or early stopping). The primary metrics are $\text{mAP}@0.5$, $\text{mAP}@0.5:0.95$, per-class precision, and per-class recall. Following [2] and [10], box-level precision and recall at the standard 0.5 IoU threshold are also reported. A normalized confusion matrix is generated to diagnose which class pairs the model conflates and where the residual error concentrates.

E. Latency Benchmarking

Throughput is measured by running 200 single-image forward passes after a 20-pass warmup on each backend. Mean latency, 95th-percentile latency, and derived FPS are reported. All measurements use 640×640 input and batch size one to reflect the streaming-inspection use case. Batch size one is mandatory for streaming deployment even though

larger batches offer better throughput, because inspection systems must produce a pass-or-fail decision per board within a fixed cycle time.

F. Deployment Considerations

Three practical considerations inform the implementation. First, the choice of 640×640 input resolution balances defect visibility (larger images help small defects) against latency (larger images quadratically increase compute). Second, ONNX export with simplification reduces the graph to a form that runs on any ONNX Runtime target without requiring framework-specific custom operators. Third, the pipeline is structured so that new weights are dropped into the same export path without code changes, which matches the continuous-integration pattern used in production ML systems.

V. RESULTS AND DISCUSSION

A. Overall Detection Performance

Table I reports the aggregate detection metrics on the DeepPCB test split. Vision-Scan reaches a mean average precision of 0.983 at the IoU threshold of 0.5, which is competitive with published YOLO-family baselines on this benchmark. Training was set for 100 epochs but early stopping triggered at epoch 50, with the best validation checkpoint recorded at epoch 20; the trainer determined that the additional 30 epochs of patience produced no further improvement. The total training time was approximately 59 minutes on the T4 GPU.

The gap between $\text{mAP}@0.5$ (0.983) and the stricter $\text{mAP}@0.5:0.95$ (0.627) follows the expected pattern for small-object detection: high accuracy at the lenient IoU threshold, with progressive degradation as the overlap requirement tightens. This pattern is consistent across the six classes and across published YOLO-family results on PCB datasets, indicating that the bounding boxes the model produces are well-localized at coarse IoU thresholds but have residual error at the pixel-precise IoU thresholds that the stricter metric averages over.

TABLE I

Aggregate Detection Metrics on DeepPCB Test Split

Metric	Value
$\text{mAP}@0.5$	0.983
$\text{mAP}@0.5:0.95$	0.627
Precision (mean)	0.979
Recall (mean)	0.948

Measured on 225 test images containing 1,498 ground-truth defect boxes.

B. Per-Class Breakdown

Table II decomposes performance across the six DeepPCB defect classes. All six classes reach $\text{mAP}@0.5$ above 0.96, with the best-performing classes being pin-hole

(0.993) and copper (0.992) and the worst-performing class being short (0.965). The narrow spread of 0.028 between the best and worst class mAP indicates that the augmentation strategy produced well-balanced training signal across the taxonomy. The short class shows the lowest recall (0.909), suggesting that some short-circuit defects are being missed at the confidence threshold used for the final metric calculation.

TABLE II

Per-Class Precision, Recall, and mAP on DeepPCB

Class	Precision	Recall	mAP@0.5
open	0.974	0.960	0.987
short	0.964	0.909	0.965
mousebite	0.992	0.966	0.984
spur	0.972	0.934	0.976
copper	0.988	0.958	0.992
pin-hole	0.986	0.960	0.993

Per-class metrics from the test-split evaluation. All six classes exceed 0.96 mAP@0.5.

The Ultralytics training pipeline also produces a normalized confusion matrix saved to the run directory. Inspection of this matrix reveals a striking pattern: the diagonal entries (correct classifications) range from 0.96 to 0.98, while almost all off-diagonal entries between defect classes are below 0.02. This means that the model very rarely confuses one defect type for another. The exception is a small 0.02 confusion between short and open, which is plausible given that both classes describe trace-level circuit faults and can resemble each other under certain layouts.

Table III reproduces the diagonal values and the background row from the normalized confusion matrix; these are the two slices of the matrix that carry the diagnostic information for this work. The diagonal column reports the per-class true-positive rate, and the background column reports the fraction of ground-truth defects that the model predicts as background (i.e. fails to detect at all).

TABLE III

Diagonal and Background Slices of the Normalized Confusion Matrix

Class	Diagonal (correct)	Predicted background
open	0.96	0.14
short	0.98	0.14
mousebite	0.97	0.21
spur	0.97	0.17
copper	0.97	0.14
pin-hole	0.98	0.19

The diagonal column shows the per-class true-positive rate when the model produces a prediction. The background column shows the fraction of ground-truth defects that the model misses entirely.

C. Inference Latency

Table IV reports per-image latency and derived throughput across two deployment backends. The PyTorch FP32 configuration on the NVIDIA T4 GPU achieves 99.3 frames per second, comfortably exceeding the 30 FPS threshold that production conveyor inspection systems typically require. The ONNX Runtime measurement is reported on CPU because the Kaggle environment did not expose a working CUDA execution provider for ONNX Runtime in the version of the runtime available at the time of evaluation; the 6.2 FPS figure therefore represents a CPU-only baseline and substantially understates the throughput that ONNX with proper GPU acceleration would deliver in production.

TABLE IV

Inference Latency and Throughput (640×640, batch size 1)

Backend	Latency mean (ms)	Latency p95 (ms)	FPS
PyTorch FP32 (T4 GPU)	10.07	10.31	99.3
ONNX Runtime (CPU)	160.15	165.51	6.2

Measured on Kaggle T4 with 20 warmup passes and 200 timed passes. The ONNX result is CPU-only due to environment limitations; production deployment with onnxruntime-gpu and proper CUDA support would be expected to match or exceed the PyTorch GPU figure.

D. Comparison to Prior Work

Table V places Vision-Scan's results in context against published detection systems evaluated on PCB defect data. Adibhatla et al. [2] report mAP near 0.98 on a proprietary dataset with their YOLOv2-based system, which establishes the upper end of the YOLO-family performance band on PCB-like imagery. Vision-Scan's 0.983 mAP@0.5 lands in the same range, validating that the architectural improvements YOLOv8 makes over its predecessors are preserved when the training set is small. Direct comparison against published YOLOv5 and YOLOv6 results on DeepPCB would require running those baselines under identical train/test split conditions, which was outside the scope of this work; those entries are marked accordingly.

TABLE V

Comparison Against YOLO-Family Baselines on PCB Defect Data

Method	Backbone	mAP@0.5
Adibhatla et al. [2]	YOLOv2	~0.98
YOLOv5s [4]	CSPDarknet	not measured

Method	Backbone	mAP@0.5
YOLOv6-S [10]	EfficientRep	not measured
Vision-Scan (ours)	YOLOv8s + aug	0.983

Published numbers depend on train/test split details in each source.

E. Discussion

Three observations stand out from the experimental results. First, the augmentation strategy combined with transfer learning was sufficient to reach competitive accuracy on a small dataset: 1,050 original training images expanded to 4,194 produced a model whose worst-class mAP@0.5 (0.965 for short) still exceeds the best results reported by some earlier-generation systems on similar tasks. Second, the early-stopping behavior is informative — the trainer found its best checkpoint at epoch 20 and could not improve over the next 30 epochs, suggesting that the dataset size and architecture are well-matched and that further training would not have helped without changing the data or model. Third, the per-class disparity is modest (0.028 between best and worst), which indicates that augmentation produced balanced gains across the taxonomy rather than disproportionately helping the easier classes.

Inspection of the normalized confusion matrix reveals where residual error actually concentrates, and the answer is not class-to-class confusion. Off-diagonal entries between defect classes are uniformly small — under 0.02 in nearly all cells. The dominant residual error is missed detection: the background row of the confusion matrix shows that 0.14 to 0.21 of the ground-truth defects are predicted as background entirely, with mousebite (0.21) and pin-hole (0.19) being the most affected. This is a recall problem rather than a precision problem, which matches the per-class numbers in Table II where every class shows precision above 0.96 but recall as low as 0.909 for short. In production terms, the model very rarely raises a false alarm but does miss some real defects.

This finding suggests two specific avenues for improvement. First, lowering the confidence threshold at inference time would surface more of the missed defects at some cost in precision; this tradeoff is operationally configurable and does not require retraining. Second, increasing input resolution from 640×640 to 1280×1280 would give the network more pixels to work with on the smallest defects, which are most likely the ones being missed; the latency cost of this change is roughly four-fold and would have to be evaluated against the throughput requirements of the target deployment. Both directions are quantifiable extensions of the current pipeline rather than fundamental architectural changes.

VI. CONCLUSION AND FUTURE WORK

Vision-Scan demonstrates that a compact three-stage pipeline — synthetic augmentation, YOLOv8 fine-tuning, and ONNX export — delivers a PCB defect detection system

that meets both the accuracy and latency requirements of production inspection. On the DeepPCB benchmark, the system reaches 0.983 mAP@0.5 with all six classes above 0.96, mean precision of 0.979, and mean recall of 0.948. Inference at 99 FPS on a T4 GPU exceeds typical conveyor-speed throughput requirements. The system is reproducible from public dependencies and open data, and the entire pipeline runs end-to-end on a single Kaggle GPU notebook in under three hours.

Several directions extend the work. First, expanding the synthetic augmentation stage with generative models such as diffusion-based defect synthesis would produce training data that better captures the long tail of rare defect geometries. Second, knowledge distillation from a larger teacher model into the deployed YOLOv8s student could raise accuracy without increasing deployment cost. Third, INT8 quantization calibrated on a representative subset of production imagery would push inference latency below the ten-millisecond mark suitable for high-speed pick-and-place lines. Fourth, evaluating the system on a second public dataset such as MVTec AD [14] would quantify cross-dataset generalization. Finally, extending the detector with a segmentation head would provide pixel-level defect localization for downstream analytics and rework routing.

Beyond these technical extensions, the broader contribution of this work is methodological. Industrial machine vision research often suffers from a gap between proof-of-concept demonstrations on small benchmarks and deployment-ready systems with characterized latency and reproducibility properties. Vision-Scan is structured to close that gap on a single project: every stage produces inspectable artifacts, every metric is reported with the script that generated it, and the entire pipeline runs from public dependencies on a public dataset. The recall-versus-precision pattern observed in the confusion matrix — high precision, with most error concentrated in missed detections rather than misclassifications — is also a useful finding for downstream work, because it points to specific improvement avenues (lower confidence thresholds, higher input resolution) that can be evaluated without redesigning the system.

Working through this project taught me several practical lessons about applied machine learning. The dataset preparation step took longer than I expected, especially handling the DeepPCB annotation format where defect labels live in sibling folders rather than next to the images; this is the kind of detail that does not appear in published papers but consumes real time in any reproduction effort. Training itself was relatively fast once the data was correctly formatted, and the early-stopping behavior at epoch 50 confirmed that the dataset was small enough that further training would not have helped. The most surprising result was that the smallest defect classes did not turn out to be the hardest to detect — pin-hole and copper reached the highest mAP@0.5 among the six classes, contradicting my initial assumption. Reading the confusion matrix carefully also

changed how I understood the residual error: most of it is missed detection (the background row) rather than confusion between defect classes (the off-diagonal entries), which means the right improvement direction is recall rather than classifier sharpening. These observations would not have been visible from the aggregate mAP number alone, and reinforce the value of producing diagnostic artifacts at every stage of the pipeline.

ACKNOWLEDGMENTS

The author thanks Professor Sanghoon Lee for guidance throughout the CS 7367 Machine Vision course. The DeepPCB dataset by Tang et al. [9] enabled the experimental evaluation reported in this paper, and the COCO dataset [15] provided the pretrained weights used as the starting point for transfer learning. The Ultralytics team's open-source YOLOv8 implementation [13] and the Albumentations library [11] formed the technical backbone of the pipeline. AI assistance from Anthropic's Claude was used to scaffold the initial code structure and report outline; all experimental work, results, code execution, and final writing were completed by the author.

REFERENCES

- [1] T. Czimmermann, G. Ciuti, M. Milazzo, M. Chiurazzi, S. Roccella, C. M. Oddo, and P. Dario, "Visual-based defect detection and classification approaches for industrial applications — a survey," *Sensors*, vol. 20, no. 5, p. 1459, 2020.
- [2] V. A. Adibhatla, H.-C. Chih, C.-C. Hsu, J. Cheng, M. F. Abbod, and J.-S. Shieh, "Defect detection in printed circuit boards using You-Only-Look-Once convolutional neural networks," *Electronics*, vol. 9, no. 9, p. 1547, Sep. 2020.
- [3] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.
- [4] G. Jocher, "Ultralytics YOLOv5," 2020. [Online]. Available: <https://github.com/ultralytics/yolov5>.
- [5] D. Tabernik, S. Šela, J. Kuščer, and D. Skočaj, "Segmentation-based deep-learning approach for surface-defect detection," *J. Intell. Manuf.*, vol. 31, no. 3, pp. 759–776, 2020.
- [6] Y. Liu, K. Xu, J. Xu, and J. Song, "A review of CV-based defect detection for metal surface," *Sensors*, vol. 23, no. 10, p. 4963, 2023.
- [7] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009.
- [8] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Berlin, Germany: Springer, 2022.
- [9] S. Tang, F. He, J. Huang, and J. Han, "Online PCB defect detector on a new PCB defect dataset," *arXiv preprint arXiv:1902.06197*, 2019.
- [10] C. Li et al., "YOLOv6: A single-stage object detection framework for industrial applications," *arXiv preprint arXiv:2209.02976*, 2022.
- [11] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, "Albumentations: Fast and flexible image augmentations," *Information*, vol. 11, no. 2, p. 125, 2020.
- [12] ONNX Runtime contributors, "ONNX Runtime: cross-platform, high-performance scoring engine for ONNX models," 2024. [Online]. Available: <https://onnxruntime.ai>.
- [13] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [14] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD — A comprehensive real-world dataset for unsupervised anomaly detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9592–9600.sss
- [15] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.
- [16] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.
- [17] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2980–2988.
- [18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [19] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 91–99.
- [20] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. European Conf. Computer Vision (ECCV)*, 2020, pp. 213–229.